ПРИМЕНЕНИЕ СОВРЕМЕННЫХ ТЕХНОЛОГИЙ, ПОДХОДОВ И ШАБЛОНОВ ПРОЕКТИРОВАНИЯ ПРИ РЕАЛИЗАЦИИ МАСШТАБИРУЕМОЙ КЛИЕНТ-СЕРВЕРНОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Костиков Ю.А., Павлов В.Ю., Романенков А.М., Синицын И.В., Терновсков В.Б.

В современном мире постоянно растет потребность в различного рода программных продуктах и технологиях, значительно упрощающих обработку данных и, как следствие, растут требования к квалификации рядовых операторов ЭВМ. Вместе с этим возрастают и потребности пользователей: приложения должны работать быстро, быть понятными и удобными с точки зрения пользовательского интерфейса, а также надежными в плане безопасности.

Одновременно с этим для разработчиков программного обеспечения важно, чтобы приложение было легко поддерживаемым и модифицируемым, с сохранением понятной архитектуры и архитектуры.

Программное обеспечение, разработанное в 2000 годах, которое отвечает и за формирование внешнего вида, и за эффективные вычисления, и за доступ к данным, уже не отвечает современным требованиям и признается устаревшим. Такие приложения перестают поддерживаться, и коммерческие фирмы вынуждены от них отказываться. Для удовлетворения вышеописанных современных требований к программному обеспечению сейчас повсеместно применяется клиент-серверная архитектура, позволяющая оптимально сбалансировать требования к быстродействию, удобству, надёжности и безопасности. Как следует из названия, эта архитектура разделяет обязанности приложения, выделяя две стороны: клиент и сервер.

Функциональные обязанности клиентского приложения заключаются в формировании запросов к серверному приложению, получения ответа от серверного приложения и обеспечение представления результатов в формате, удобном для пользователя.

Обязанности серверного приложения заключаются в получении запросов от клиентского приложения и интерпретации их, выполнении запросов к базе данных, отправку результатов клиентскому приложению, обеспечению разграничения доступа к данным, поддержание стабильного многопользовательского режима.

Серверное веб-приложение

В мире программных продуктов происходят заметные изменения. В последние годы наибольшей популярностью начинают пользоваться веб-приложения, которые развились от первоначального набора статических html-страниц до полноценных приложений, по своей сложности сравнимых с классическими оконными приложениями.

Тем не менее, сохраняется ряд идеологических отличий, которые заключаются в следующем: в использовании транспортного протокола и реального сетевого расположения, а также в отсутствии состояния, то есть физического конкретного экземпляра приложения.

HTTP - протокол прикладного уровня для передачи данных в гипертекстовом формате - был представлен в 1992 году. В современном мире HTTP и его защищенный аналог HTTPS стал де факто стандартом взаимодействия между приложениями. Основополагающей моделью взаимодействия является схема запрос-ответ. Для каждого запроса клиентского приложения с серверной стороны возвращается ответ. Ключевая особенность для HTTP и всей веб-разработки в целом заключается в отсутствии состояния: веб-сервер может обрабатывать тысячи запросов от тысяч клиентов в секунду, причем веб-сервер не делает никаких отличий между различными клиентами, в отличие от классических приложений, где каждое ориентировано на конкретного пользователя. Все данные, которыми оперирует веб-приложение, хранятся на серверах баз данных, а доступ к ним контролируется с помощью систем управления базой данных (СУБД).

Для организации обмена данными и сообщениями между приложениями и между модулями приложений весьма эффективно используется REST архитектура.

Архитектура REST (REpresentational State Transfer) – стиль архитектуры программного обеспечения для распределенных систем, таких как World Wide Web, который, как правило, используется для построения веб-служб. В общем случае REST является очень простым интерфейсом управления информацией без использования каких-то дополнительных внутренних прослоек. Каждая единица информации однозначно определяется глобальным идентификатором, таким как URL. Каждая URL в свою очередь имеет строго заданный формат. Для HTTP-протокола действие над данными задается с помощью методов:

1. GET (получить)
2. PUT (добавить, заменить)
3. POST (добавить, изменить, удалить)
4. DELETE (удалить)

Схема взаимодействия между клиентом и сервером в архитектуре REST представлена на рис. 1.

Опишем предлагаемую реализацию REST взаимодействия. Будем считать, что ресурс представляет собой один или несколько экземпляров класса, реализованного на языке C# 5.0.[1] Действия над ресурсом определены с помощью методов интерфейса языка C# 5.0. Отправка запроса разбивается на несколько стадий:

1. Авторизация на сервере
2. Формирование URL и отправка HTTP-запроса

3. Формирование тела ответа (в случае Post / Put) на сервере и отправка HTTP-ответа

4. Преобразование ответа в клиентском приложении и его интерпретация.

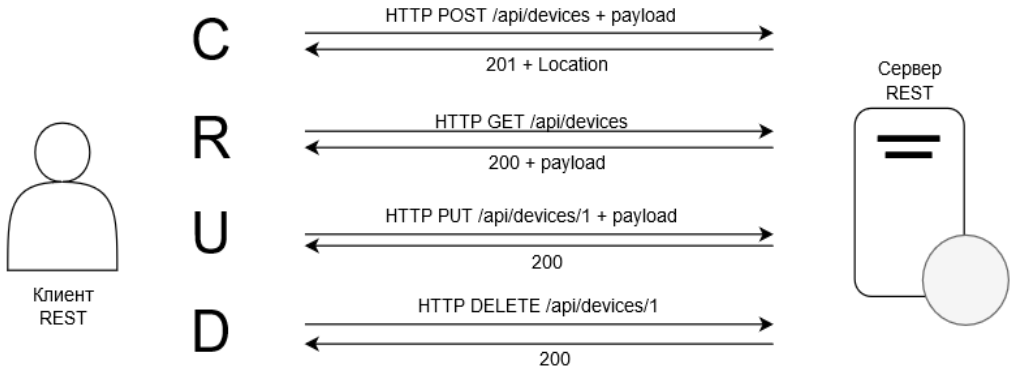


Рис. 1. Схема взаимодействия между клиентом и сервером в архитектуре REST

Далее организуем такой способ взаимодействия, который бы позволил один раз реализовать всю логику обращения к веб-серверу и интерпретацию ответов, а после использовать ее для каждого ресурса.

Основой данного решения является тот факт, что все методы взаимодействия являются однотипными, исходя как из их логики, так и из их программной реализации. Отличительными чертами каждого запроса являются:

1. Метод протокола HTTP (Get, Post, Put, Delete)
2. Имя контроллера веб-приложения, обрабатывающего запрос
3. Метод контроллера
4. Параметры запроса

В связи с этим можно определить некий один общий, базовый, метод, в который будут передаваться характеристики, и тот на их основании будет формировать запрос и принимать ответ.

Такое поведение позволяет реализовать библиотека Castle Dynamic Proxies, с помощью которой можно динамически генерировать классы на основе интерфейсов, при этом подставляя в реализацию каждого метода класса предопределенный код.

Создание и отправка HTTP-запросов происходят с помощью стандартных средств языка C# 5.0, но перед этим необходимо получить параметры запроса. Первые три удобно представить в виде атрибутов, при этом первый (метод протокола HTTP) будет применяться ко всему интерфейсу ресурса, а остальные два (имя и метод контроллера) будут относиться к каждому отдельному методу. Параметры запроса передаются в параметры метода.

Возникает следующая проблема: очевидно, что методы класса ресурса должны работать с самим ресурсом: принимать в качестве аргумента и возвращать из метода его результат, но в обобщенной генерации запроса ничего не известно о типе ресурса. В связи с этим используется рефлексия, позволяющая динамически во время выполнения приложения получать информацию о типах данных и непосредственно сами типы данных.

Предлагаемый алгоритм работы:

1. Определение базового поведения для взаимодействия со всеми ресурсами
2. Представление запроса в виде сообщения протокола HTTP
3. Отправка запроса веб-серверу
4. Получение ответа
5. Извлечение данных из сообщения
6. Определение интерфейса, реализующего конкретный ресурс, и его методы
7. Динамическое создание класса на языке C#, реализующего интерфейс ресурса
8. Вызов соответствующих методов для доступа к ресурсам

Ниже приведен пример кода на языке C# 5.0. интерфейса ресурса.

```
[HttpResource("someresource")]
public interface ISomeResourceClient
{
    [HttpMethod("list")]
    [HttpRequest(HttpRequestType.Get)]
    Task<IList<SomeResource>> ListAsync();

    [HttpMethod]
    [HttpRequest(HttpRequestType.Get)]
    Task<SomeResource> GetAsync(int id);

    [HttpMethod]
    [HttpRequest(HttpRequestType.Put)]
    Task CreateAsync(SomeResource resource);

    [HttpMethod]
    [HttpRequest(HttpRequestType.Post)]
    Task UpdateAsync(SomeResource resource);
}
```

Организация работы с СУБД

Доступ к базе данных является одной из популярных тем для обсуждения в области разработки программного обеспечения. На протяжении долгого времени, подавляющей популярностью пользовался шаблон ActiveRecord [2]. Структура данного шаблона представлена на рис. 2.

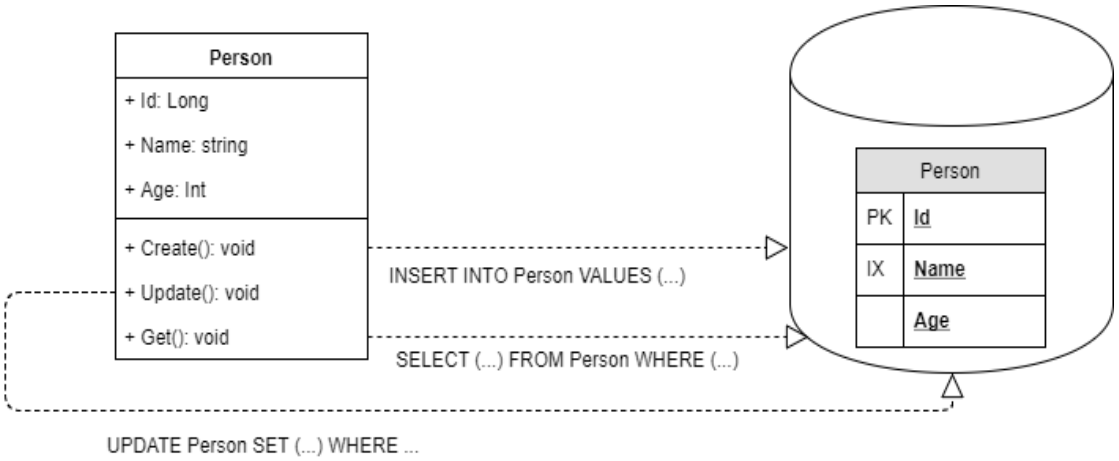


Рис. 2. Структура шаблона ActiveRecord

Суть шаблона сводится к следующему: каждый объект класса представляет собой обертку над строкой из соответствующей таблицы, содержит в себе поля, однозначно сопоставимые с колонками таблицы, и набор методов, каждый из которых генерирует запросы или команды на языке SQL. Подход хорошо работает, когда связи между таблицами вида многие-ко-многим и один-ко-многим отсутствуют или сведены к минимуму, а также набор полей сущности невелик. К тому же, динамическое составление SQL запросов чревато ошибками и потенциальной угрозой безопасности данных приложения, например, SQL-инъекции. (SQL Injection)

В противном случае используется альтернатива - технология объектно-реляционного сопоставления (ORM), которая позволяет отделить и вынести всю логику, специфичную для генерации запросов SQL в отдельные классы, а в самих сущностях оставить лишь данные и бизнес логику, специфичную для приложения. Графическая модель ORM представлена на рис. 3.

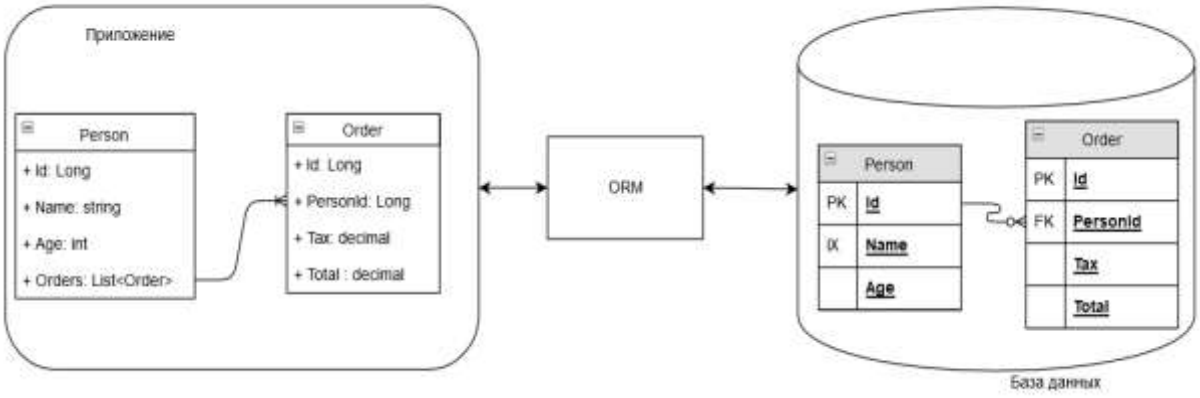


Рис. 3. Графическая модель ORM

Также технология ORM облегчает написание операторов соединения (JOIN), группировки (GROUP BY, HAVING), фильтрации (WHERE) и сор-

тировку (ORDER BY) предоставляя удобный и объектно ориентированный способ составления динамических запросов [3, 4]. Некоторые реализации ORM поддерживают систему кеширования и встроенную защиту от инъекций SQL кода, направленных на порчу или неконтролируемый доступ к данным, путем экранирования параметров запроса при его составлении. С точки зрения реализации, технологии ORM основана на совместном использовании двух шаблонов проектирования [2]: Repository (Репозиторий) и UnitOfWork (Единица работы). “Репозиторий” служит посредником (Mediator), задача которого - отделить логику приложения от логики доступа к данным, предоставляя интерфейс, подобный обычной коллекции в памяти, для вызывающего кода.

Проблема, которую решает шаблон “Единица работ” - это контроль состояния объектов и всех изменений над объектами, полученных путем чтения данных с БД. Такой контроль позволяет объединять отправляемые команды SQL в пакеты команд и исполнять их на сервере баз данных под определенным уровнем изоляции, гарантируя консистентность данных.

Архитектура CQRS

Рассмотрение данной архитектуры можно начать с принципа CQS (Command-Query-Separation), основная идея которого заключается в том, что в объекте методы могут быть двух типов:

1. Команды: изменяют состояние объекта, не возвращая результатов.
2. Запросы: возвращают результат, не изменяя состояние объекта. Другими словами, запросы никоим образом не влияют на состояние объекта, не модифицируют его явно или неявно.

На рис. 4 показана схема принципа CQS.

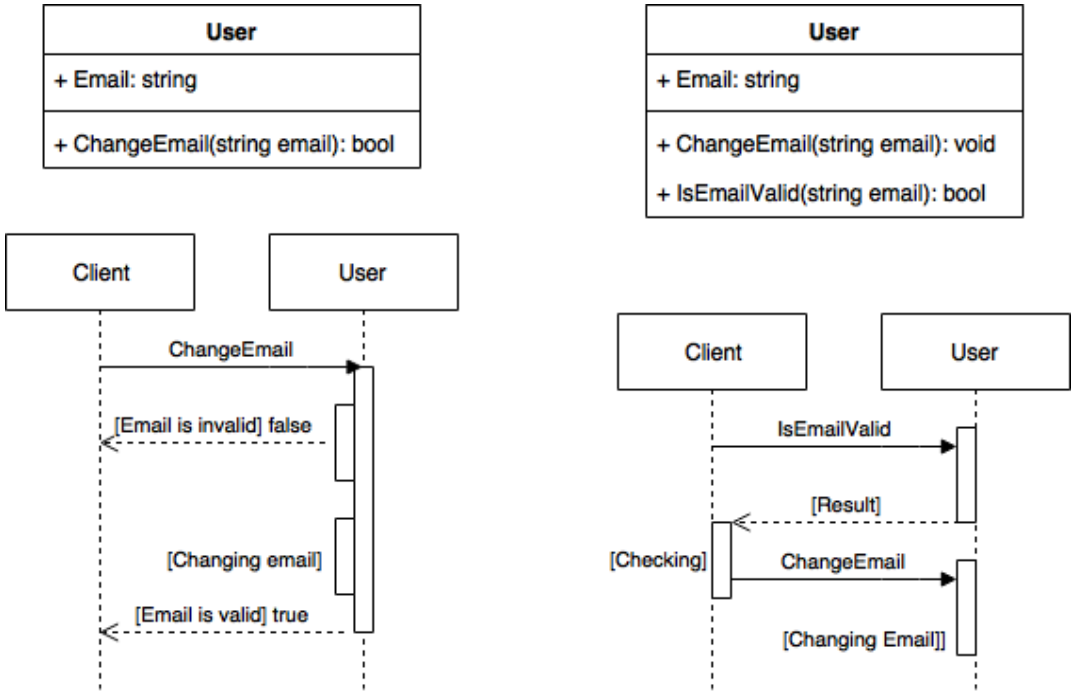


Рис. 4. Схема принципа CQS

Такое разделение методов класса облегчает поддержку и сопровождение кода, позволяя выносить все проверки состояния и покрывать их автоматическими тестами.

Похожая идея (CQRS) разделения ответственностей лежит на уровне проектирования архитектуры приложения, но в этом случае понимается не уровень методов объекта, а уровень объектов системы. Таким образом, глобальная архитектура системы имеет вид, представленный на рис. 5.

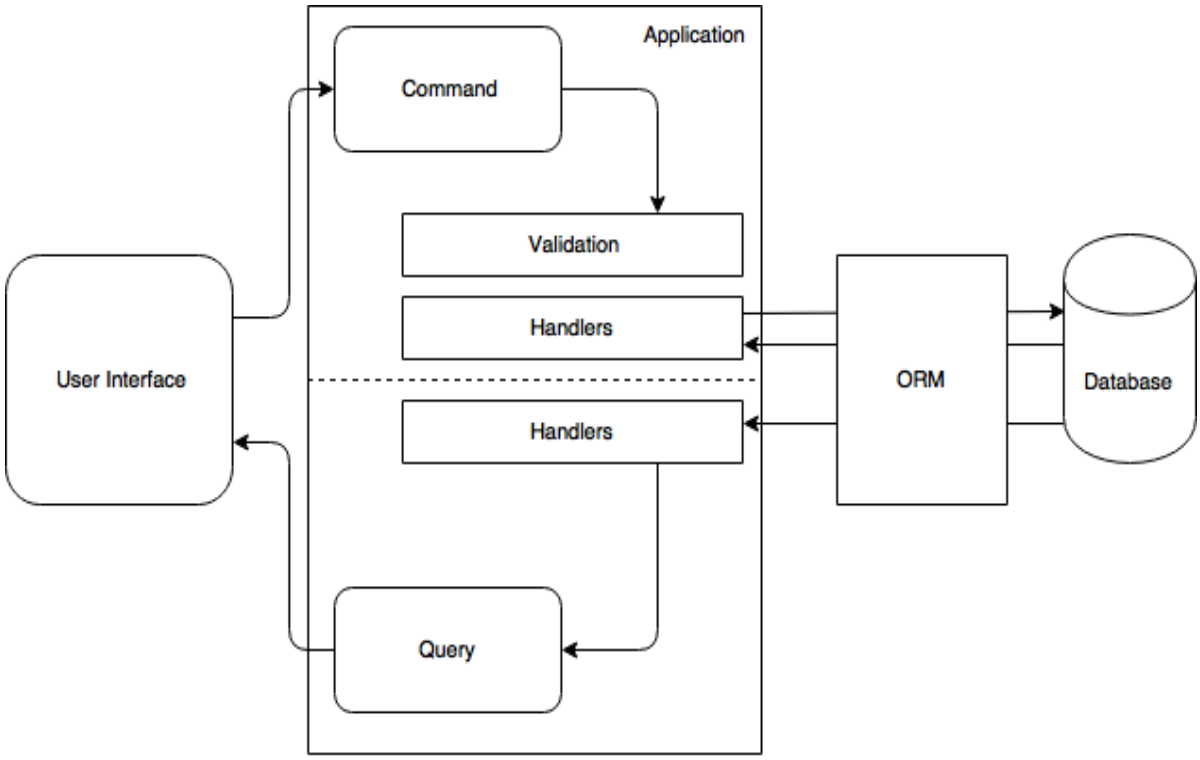


Рис. 5. Вид глобальной архитектуры системы

В такой архитектуре явно разделяются модель исполнения запросов и модель исполнения команд. С каждым действием системы связан класс (класс команды), содержащий все необходимые данные для выполнения этого действия. С каждым классом команды связан класс-обработчик действия. Такое же описание справедливо и для запросов.

В контексте языка программирования C# .NET Framework 4.6.2 данное описание может быть выражено следующими базовыми интерфейсами:

Тогда для осуществления всех действий и запросов создаются классы, реализующие заданные интерфейсы, причем каждый такой класс будет сосредоточен на реализации конкретной задачи, сохраняя принцип единственной ответственности (SRP).

Весь скользящий функционал, связанный с логированием, обработкой ошибок, измерением производительности и т.д. реализуется с помо-

щью дополнительного шаблона проектирования “Декоратор” (Decorator) [2]:

Задачи конструирования объектов реализации, проверка прав доступа и валидирование входных параметров реализуются с помощью отдельных шаблонов проектирования - “Фасад” (Facade) и “Фабричный метод” (FactoryMethod).

Применение продемонстрированных техник обеспечивают надежную, расширяемую и поддерживаемую архитектуру.

Отдельно необходимо отметить возможности дальнейшего масштабирования производительности. Существует несколько техник:

1. Оптимизация структуры базы данных: добавление индексов, материализованных представлений на ускоренный доступ чтения данных.

2. Использование более легковесных конструкций для взаимодействия с базой данных

3. Разделение единого SQL хранилища: использование технологий кэширования и NoSQL.

В первом случае масштабирование нагрузки производится за счет базы данных, и код приложения не будет затронут. В остальных случаях гибкая архитектура позволяет заменять отдельные команды и запросы их оптимизированными аналогами, не совершая существенные изменения в архитектуре веб-приложения.

В работе предложен вариант взаимодействия на основе архитектуры REST между компонентами программной системы. Описана и продемонстрирована методика разработки приложений для взаимодействия с СУБД на основе современных паттернов проектирования. Приведены конкретные рекомендации и примеры кода для реализации гибкого расширяемого программного комплекса. Рассмотренный подход является актуальным в настоящее время и позволяет разрабатывать легко поддерживаемое программное обеспечение.

Литература

1. Эндрю Троелсон. Язык программирования C# 5.0 и платформа .NET 4.5. Изд. Вилльямс. 2015.

2. Эрих Гамма, Ричард Хелм, Ральф Джонсон, Джон Влиссидес. Приемы объектно-ориентированного проектирования. Паттерны проектирования. Изд. Питер. 2016.

3. К. Дж. Дэйт. Введение в системы баз данных. Изд. Вилльямс. 2017.

4. Ицик Бен-Ган. Microsoft SQL Server 2012. Высокопроизводительный код T-SQL. Оконные функции. Русская Редакция, БХВ-Петербург. 2013.